MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE LA RECHERCHE ET DE L'INNOVATION
****
ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU
****
INSTITUT DU GÉNIE INFORMATIQUE ET DES TÉLÉCOMMUNICATIONS IGIT



BURKINA FASO
****
La Patrie ou la Mort, nous Vaincrons

Rapport de travaux pratiques

TP 1 À 3 : CHIFFREMENT AUTHENTIFIÉ, PKI À DEUX NIVEAUX ET CANAL TLS MUTUEL

Réalisé par :

LOARI Moumouni Boukary

Filière :

Première Année Génie Informatique

Encadrant

Dr. Zakaria Sawadogo

Année universitaire 2026–2027

1. Introduction et environnement de travail

Ce rapport présente la réalisation et la correction des trois travaux pratiques du module EC1 : un mini-coffre de fichiers (TP 1), une infrastructure à clé publique à deux niveaux avec OpenSSL (TP 2), puis le déploiement d'un canal TLS avec authentification mutuelle, sa supervision et son renouvellement automatique (TP 3). Conformément à la règle d'ingénieur du sujet, aucun algorithme n'est réimplémenté : on assemble correctement des briques standard.

2. TP 1 : Chiffrement authentifié et dérivation de clés

Le module `coffre.py` chiffre un fichier avec un mot de passe : Argon2id dérive une clé maîtresse, HKDF-SHA256 en tire une sous-clé par usage, et AES-256-GCM chiffre et authentifie les données.

2.1 Étape 1 : Dérivation des clés

La fonction `mesurer_parametres` chronomètre une dérivation.

Les durées varient d'une mesure à l'autre (la première inclut l'initialisation) : il faut répéter la mesure et ajuster `T` et `M` jusqu'à environ 0,5 s sur la machine cible. Le jeu retenu est (4, 256 Mio, 4).

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp1]
└─$ python3 -c "import coffre; print(coffre.mesurer_parametres(4, 256*1024, 4))"

1.670787056998961

┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp1]
└─$ python3 -c "import coffre; print(coffre.mesurer_parametres(4, 256*1024, 4))"

0.49222455699782586

┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp1]
└─$ python3 -c "import coffre; print(coffre.mesurer_parametres(4, 256*1024, 4))"

0.4872763009989285
```

2.2 Étape 2 : Format et fonctions chiffrer / déchiffrer

L'en-tête de 44 octets est authentifié sans être chiffré :

Offset	Taille	Champ
0	4 octets	Identifiant de format CFR1
4	12 octets	t, m, p : trois entiers de 4 octets, gros-boutiste
16	16 octets	Sel Argon2id
32	12 octets	Nonce AES-GCM
44	variable	Chiffré suivi de l'étiquette de 16 octets

- chiffrer tire un sel et un nonce neufs avec os.urandom à chaque appel.
- Les données associées (AAD) sont l'en-tête complet suivi du nom du fichier encodé en UTF-8.
- dechiffrer relit t, m, p dans l'en-tête, redérive la clé avec ces valeurs et lève FormatInconnu si l'identifiant ou la taille est incorrect. Un échec d'authentification lève InvalidTag.

2.3 Étape 3 : Preuves par les tests

Huit tests pytest (les cinq demandés plus trois) passent tous : 8 passed. Pour que la suite reste rapide, les tests abaissent T, M et P ; comme le format relit les paramètres dans l'en-tête, cela ne change pas ce qui est vérifié.

Test	Résultat	Propriété vérifiée
test_aller_retour	Données retrouvées	Correction
test_chiffrement_probabiliste	Deux sorties différentes (sel et nonce différents)	Chiffrement probabiliste
test_mauvais_mot_de_passe	InvalidTag	Authentification de la clé
test_octet_modifie	InvalidTag	Intégrité

Test	Résultat	Propriété vérifiée
test_nom_echange	InvalidTag	Liaison au contexte par les données associées
test_entete_modifie (en plus)	InvalidTag	Authentification de l'en-tête
test_format_inconnu (en plus)	FormatInconnu	Refus d'une version inconnue
test_rechiffrer_et_evolution_des_parametres (en plus)	Réussi	Évolution des paramètres (étape 4)



2.4 Étape 4 : Évolution des paramètres

Le format le permet parce que la version (CFR1) et les paramètres t, m, p sont écrits dans chaque fichier. On peut donc durcir T et M pour les nouveaux fichiers sans rendre illisibles les anciens, qui se déchiffrent avec leurs propres paramètres. La fonction rechiffrer(ancien_mdp, nouveau_mdp, nom, blob) déchiffre puis rechiffre avec de nouveaux sel et nonce et les paramètres courants : elle sert à changer le mot de passe comme à migrer un ancien fichier. Le test vérifie que l'ancien fichier reste lisible après changement des constantes, que le nouveau a des paramètres différents dans l'en-tête, et que l'ancien mot de passe ne lit plus le nouveau fichier.

La sortie de coffre.py donne :

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp1]
└─$ python3 coffre.py
t=1 m=32 Mio p=4 : 0.029 s
t=2 m=64 Mio p=4 : 0.065 s
t=3 m=64 Mio p=4 : 0.114 s
t=3 m=128 Mio p=4 : 0.180 s
```

2.5 Questions de réflexion

1. **Pourquoi le sel est-il stocké en clair ?** Le sel n'a pas à être secret : il rend la dérivation unique pour chaque fichier. Un même mot de passe ne donne pas la même clé partout et les tables précalculées deviennent inutiles ; l'attaquant doit refaire un calcul Argon2id coûteux par fichier et par mot de passe testé. Il faut de toute façon le connaître pour redériver la clé. Sa publication n'affaiblit donc pas la sécurité.
2. **Pourquoi pas de risque de répétition du nonce ?** Le danger dans GCM est de réutiliser le même couple (clé, nonce). Ici la clé dépend d'un sel de 128 bits tiré au hasard pour chaque fichier : deux fichiers ont des clés différentes avec une probabilité écrasante, donc leurs nonces ne se rencontrent jamais sous la même clé. Chaque clé ne chiffre qu'un seul message. Le risque ne reviendrait que si l'on chiffrait un très grand nombre de messages avec une seule clé.
3. **Que se passerait-il si le nom n'était pas dans les données associées ?** Un attaquant pourrait échanger deux fichiers sans être détecté. Scénario : une banque stocke `virement_autorise.dat` et `brouillon.dat` chiffrés avec le même mot de passe. L'attaquant les renomme l'un en l'autre : chaque blob est intact, l'étiquette est valide, et le programme lit le brouillon en croyant lire le virement. Avec le nom en données associées, l'échange fait échouer l'étiquette (`test_nom_echange`).
4. **Pourquoi authentifier l'en-tête ?** Il contient t, m, p et le sel, lus avant le déchiffrement. S'ils pouvaient être modifiés sans détection, un attaquant pourrait réduire t et m pour imposer une dérivation faible, ou les gonfler pour épuiser la mémoire du serveur (déni de service). L'authentification garantit que l'étiquette échoue dès qu'un octet de l'en-tête change (`test_entete_modifie`). Un contrôle de plage sur t, m, p avant de lancer Argon2id reste conseillé.
5. **Pourquoi ne jamais afficher la raison précise d'un échec ?** Distinguer « mauvais mot de passe », « fichier corrompu » et « mauvais nom » donnerait à un attaquant un oracle :

il saurait quelle partie est correcte et pourrait deviner morceau par morceau. Avec AES-GCM, tous ces cas produisent d'ailleurs le même InvalidTag. Le message utilisateur reste donc unique (« déchiffrement impossible »), le détail éventuel allant dans un journal interne.

3. TP 2 : Construire une infrastructure de gestion de clés

L'infrastructure comprend une AC racine (P-384, 10 ans, hors ligne), une AC intermédiaire (P-256, 5 ans, limitée au domaine tp.local) et deux certificats finaux de 90 jours : le serveur www.tp.local et le client alice@tp.local. Le script pki.sh construit l'ensemble en une passe.

3.1 Étape 1 : Rôle des lignes d'extensions

Ligne	**Rôle**
basicConstraints = critical,CA:TRUE,pathlen:0	Le certificat est une AC, mais elle ne peut signer que des certificats finaux, pas d'autre AC
keyUsage = critical,keyCertSign,cRLSign	Sa clé ne sert qu'à signer des certificats et des CRL
subjectKeyIdentifier / authorityKeyIdentifier	Identifiants de clés qui permettent de construire la chaîne
nameConstraints = critical,permitted;DNS:.tp.local,permitted;DNS:tp.local,permitted;email:tp.local	L'intermédiaire n'est valable que pour tp.local, ses sous-domaines (.tp.local) et les courriels de tp.local ; critical : un client qui ne comprend pas cette extension doit refuser le certificat
ext_serveur : CA:FALSE, digitalSignature, extendedKeyUsage = serverAuth	Certificat final, utilisable uniquement comme serveur TLS
ext_client : idem avec clientAuth et subjectAltName = email:alice@tp.local	Utilisable uniquement comme client TLS ; l'identité d'Alice est fixée dans la configuration

Ligne	Rôle
default_md, default_crl_days (racine : SHA-384, 30 jours ; intermédiaire : SHA-256, 7 jours)	Algorithme de signature et durée de validité des CRL : la racine, hors ligne, publie rarement ; l'intermédiaire, en ligne, publie souvent
policy = politique_souple	Seul le commonName est obligatoire dans le sujet ; pays et organisation sont facultatifs
copy_extensions = copy (intermédiaire) / none (racine)	L'intermédiaire recopie les noms alternatifs de la CSR (sauf ceux déjà fixés dans la section) ; la racine n'en recopie aucune et décide seule des extensions de l'AC qu'elle signe

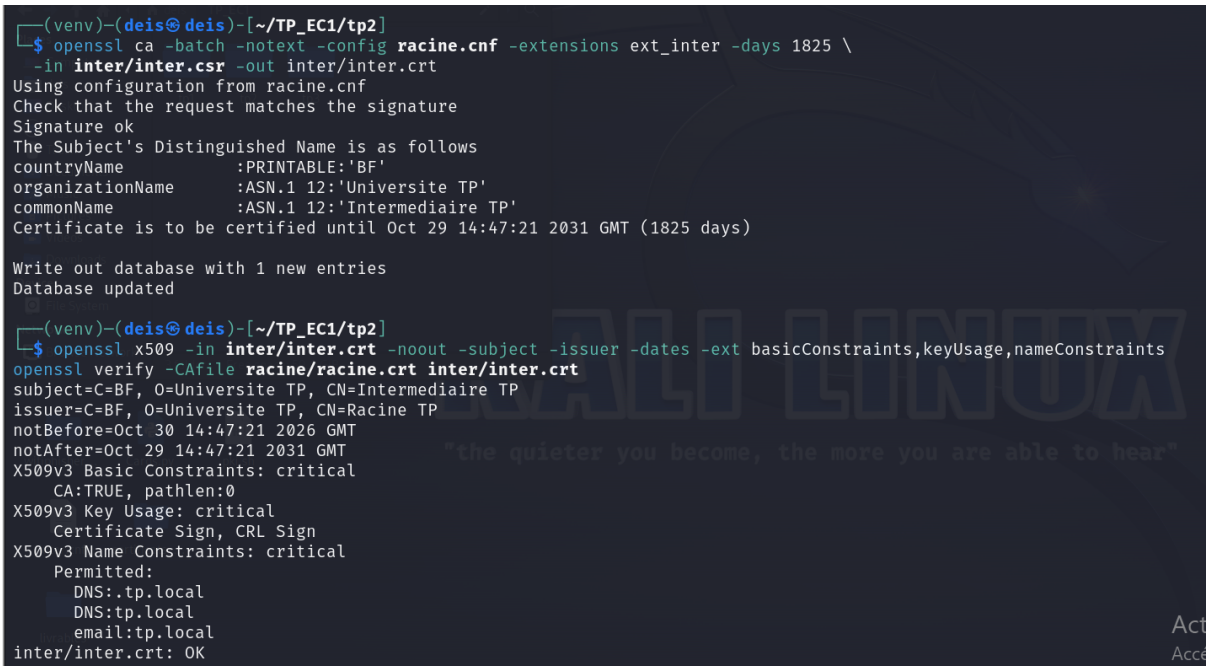
3.2 Étape 2 : AC racine

La clé P-384 est générée avec openssl genpkey, puis le certificat est auto-signé (SHA-384, 3650 jours) avec basicConstraints = critical,CA:TRUE,pathlen:1. Dans la sortie de openssl x509 -text, l'émetteur et le sujet sont identiques (C = BF, O = Universite TP, CN = Racine TP) : le certificat est **auto-signé**, la racine se garantit elle-même et c'est sa présence dans le magasin de confiance qui fait foi. pathlen:1 autorise une seule AC sous elle.



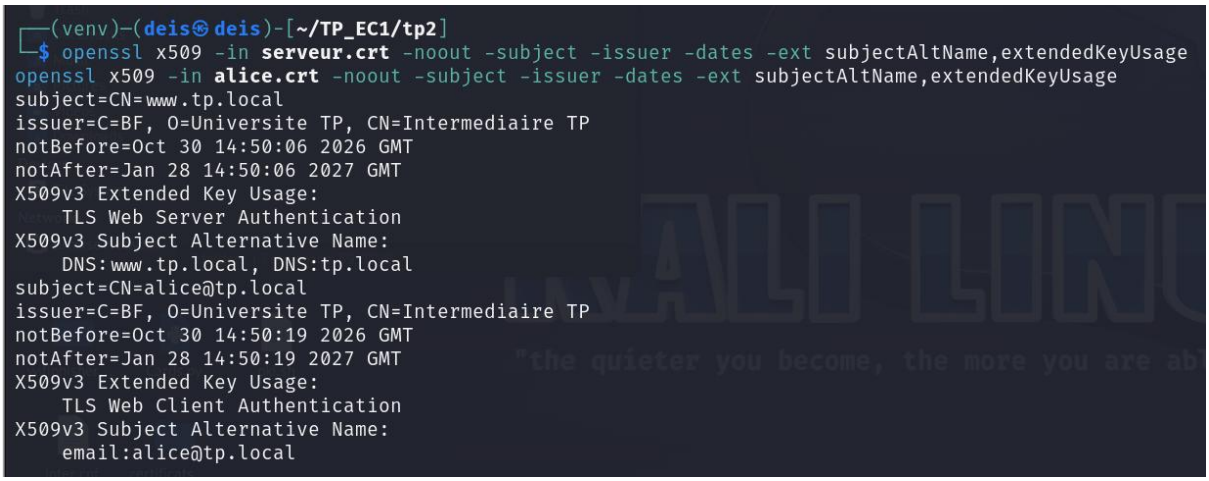
3.3 Étape 3 : AC intermédiaire

La clé P-256 et la demande sont générées, puis la racine signe avec la section ext_inter pour 1825 jours. openssl x509 confirme CA:TRUE, pathlen:0, l'usage Certificate Sign, CRL Sign et les contraintes de nom critiques DNS:.tp.local, DNS:tp.local et email:tp.local.



3.4 Étape 4 : Certificats de serveur et de client

Chaque demandeur génère sa propre clé P-256 et sa CSR ; l'intermédiaire signe pour 90 jours. Le serveur a pour sujet CN=www.tp.local et pour noms alternatifs DNS:www.tp.local et DNS:tp.local. Le client a pour sujet CN=alice@tp.local et pour nom alternatif email:alice@tp.local (fixé par ext_client). Pour Bob, pki.sh utilise ext_client_csr, qui reprend le nom alternatif de sa demande (email:bob@tp.local).



3.5 Étape 5 : Validations observées

Commande	Résultat obtenu	Interprétation
verify -verify_hostname www.tp.local serveur.crt	OK	Chaîne valide, nom présent dans le SAN
idem avec autre.example	error 62 ... hostname mismatch	Chaîne valide, mais le nom ne correspond pas
verify -purpose sslclient alice.crt	OK	Usage client autorisé
verify -purpose sslserver alice.crt (test supplémentaire)	error 26 ... unsuitable certificate purpose	Certificat clientAuth seul : refusé comme serveur
verify example.crt (www.example.com)	error 47 ... permitted subtree violation	Nom hors de tp.local

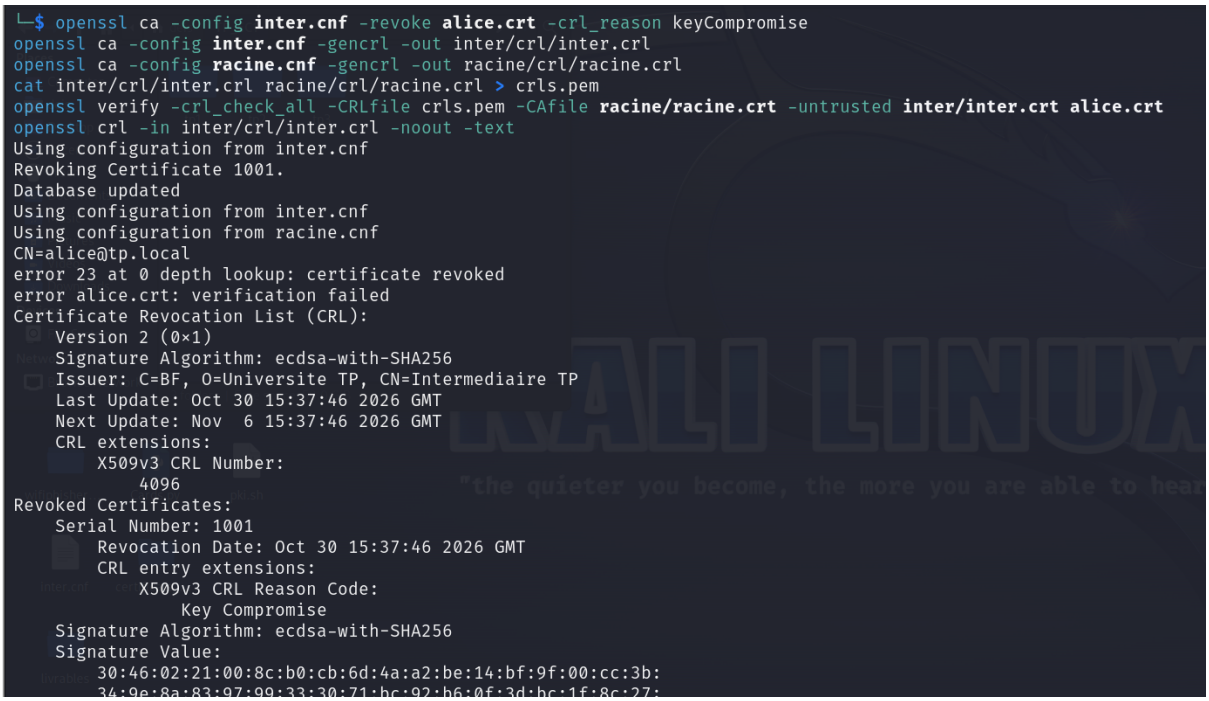
Pourquoi l'AC a-t-elle pu signer le certificat www.example.com alors qu'il est refusé à la validation ? La commande openssl ca ne contrôle pas les contraintes de nom de son propre certificat : signer est un acte mécanique de l'AC. Le contrôle est fait par celui qui **valide** la chaîne, qui applique les contraintes de l'intermédiaire à tout ce qu'elle a signé. C'est tout l'intérêt : même une intermédiaire compromise ne peut pas faire accepter un certificat pour www.example.com.

3.6 Étape 6 : Révocation

La commande openssl ca -revoke alice.crt -crl_reason keyCompromise marque la ligne correspondante d'index.txt avec l'état R (numéro de série 1001). Les CRL de l'intermédiaire et de la racine sont ensuite générées et concaténées dans crls.pem.

Vérification (-crl_check_all)	Résultat
alice.crt	error 23 ... certificate revoked
serveur.crt (non révoqué)	OK

La CRL de l'intermédiaire (openssl crl -text) indique un Next Update sept jours après le Last Update, un numéro de CRL 4096 (le compteur démarre à 1000 en hexadécimal), le numéro de série 1001 et la raison Key Compromise.



```
└─$ openssl ca -config inter.cnf -revoke alice.crt -crl_reason keyCompromise
openssl ca -config inter.cnf -gencrl -out inter/crl/inter.crl
openssl ca -config racine.cnf -gencrl -out racine/crl/racine.crl
cat inter/crl/inter.crl racine/crl/racine.crl > crls.pem
openssl verify -crl_check_all -CRLfile crls.pem -CAfile racine/racine.crt -untrusted inter/inter.crt alice.crt
openssl crl -in inter/crl/inter.crl -noout -text
Using configuration from inter.cnf
Revoking Certificate 1001.
Database updated
Using configuration from inter.cnf
Using configuration from racine.cnf
CN=alice@tp.local
error 23 at 0 depth lookup: certificate revoked
error alice.crt: verification failed
Certificate Revocation List (CRL):
    Version 2 (0x1)
    Signature Algorithm: ecdsa-with-SHA256
    Issuer: C=BF, O=Universite TP, CN=Intermediaire TP
    Last Update: Oct 30 15:37:46 2026 GMT
    Next Update: Nov  6 15:37:46 2026 GMT
    CRL extensions:
        X509v3 CRL Number:
            4096
Revoked Certificates:
    Serial Number: 1001
        Revocation Date: Oct 30 15:37:46 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Key Compromise
    Signature Algorithm: ecdsa-with-SHA256
    Signature Value:
        30:46:02:21:00:8c:b0:cb:6d:4a:a2:be:14:bf:9f:00:cc:3b:
        34:9e:8a:83:97:99:33:30:71:bc:92:b6:0f:3d:bc:1f:8c:27:
```

3.7 Étape 7 : Automatisation

pki.sh regroupe les étapes 2 à 4 avec set -euo pipefail : il s'arrête à la première erreur. Il prend en paramètre le nom du client (alice par défaut).

3.8 Questions de réflexion

1. **Pourquoi la clé de la racine doit-elle rester hors ligne ? Que faire si l'intermédiaire ou la racine est compromise ?** Tout repose sur la racine : sa clé est conservée sur un support déconnecté (idéalement un module matériel) et ne sert que rarement, pour signer une intermédiaire ou une CRL. Si l'**intermédiaire est compromise**, on la révoque depuis la racine (CRL de la racine), on en crée une nouvelle avec une nouvelle clé, puis on réémet ou révoque les certificats qu'elle a signés ; les clients n'ont rien à réinstaller. Si la **racine est compromise**, il n'y a pas de remède local : tout ce qui en dépend est discrédité, il faut créer une nouvelle racine et la redistribuer dans tous les magasins de confiance.

2. **À quoi sert pathlen:0 ? Quel risque si on l'oublie ?** Il interdit à l'intermédiaire de signer une autre AC : elle ne peut émettre que des certificats finaux. Sans cette limite, quiconque contrôle la clé de l'intermédiaire pourrait créer une AC enfant, hors politique et hors contrôle, et la chaîne resterait valide.
3. **Pourquoi la clé privée du serveur est-elle générée par le serveur et non par l'AC ?** Une clé privée ne doit exister qu'à un seul endroit et ne jamais circuler. Si l'AC la générait, elle la connaîtrait (et pourrait usurper son client) et il faudrait la transmettre par un canal sûr : deux occasions de fuite. Avec une CSR, le demandeur prouve qu'il détient la clé, car la demande est signée avec elle, et seule la clé publique voyage.
4. **CN ou SAN : lequel les clients modernes vérifient-ils ?** Le **SAN**. La RFC 6125 a rendu le CN obsolète pour les noms de domaine, et les navigateurs comme OpenSSL (`-verify_hostname`) l'ignorent dès qu'un SAN existe. Le CN est conservé pour la compatibilité et la lisibilité ; on met donc toujours le nom dans le SAN.
5. **Limites des CRL ; comparaison avec OCSP et les certificats de très courte durée.** Les CRL grossissent avec le nombre de révocations, ne sont publiées que périodiquement (ici tous les sept jours) et doivent être téléchargées : un certificat révoqué reste accepté jusqu'à la prochaine mise à jour, et un client qui ne parvient pas à les obtenir les ignore souvent (soft-fail). **OCSP** donne l'état d'un seul certificat quasiment en temps réel, mais impose une requête par connexion (délai, fuite de la navigation vers l'AC, disponibilité du répondeur) ; l'agrafage OCSP (stapling) en corrige une partie. Les **certificats de très courte durée** rendent la révocation presque inutile : une clé volée cesse d'être utilisable à l'expiration, au prix d'un renouvellement entièrement automatisé (voir le TP 3).
6. **Politique de certification minimale.** Les certificats peuvent être demandés par les administrateurs du domaine tp.local et par les utilisateurs inscrits de l'université. L'identité est vérifiée selon le cas : pour un serveur, par la preuve du contrôle du nom et pour un client, par le service informatique sur pièce justificative. Les noms délivrés sont uniquement ceux placés sous tp.local. Les certificats finaux sont valables 90 jours, l'intermédiaire 5 ans et la racine 10 ans. Un certificat sert à un seul usage, serveur ou client, et aucune feuille ne peut être une AC. La révocation est faite sur demande du titulaire et traitée sous 24 heures. La CRL de l'intermédiaire est mise à jour au moins tous les 7 jours, et immédiatement après chaque révocation. La clé de la racine reste hors ligne et sauvegardée, tandis que celle de l'intermédiaire est conservée sur un poste restreint.

4. TP 3 : Déployer un canal sécurisé et gérer les certificats

Ce TP réutilise la PKI du TP 2 pour configurer un serveur TLS 1.2 et 1.3 avec authentification mutuelle, observer la poignée de main, superviser les expirations et automatiser le renouvellement.

4.1 Étape 1 : Préparation des fichiers

Le serveur présente sa chaîne, c'est-à-dire son certificat suivi de celui de l'intermédiaire, jamais la racine.

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp2]
└─$ W=w
cat serveur.crt inter/inter.crt > serveur_chaine.crt
sudo mkdir -p /etc/nginx/tls
sudo install -m 600 serveur.key /etc/nginx/tls/serveur.key
sudo install -m 644 serveur_chaine.crt racine/racine.crt crls.pem /etc/nginx/tls/
echo "127.0.0.1 $W$W$W.tp.local" | sudo tee -a /etc/hosts
127.0.0.1 www.tp.local

┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp2]
└─$ ls -l /etc/nginx/tls/
tail -1 /etc/hosts
total 20
drwx------ 4 root root 4096 Oct 28 16:50 archive
-rw-r--r-- 1 root root  780 Oct 30 11:44 crls.pem
-rw-r--r-- 1 root root  786 Oct 30 11:44 racine.crt
-rw------- 1 root root  241 Oct 30 11:44 serveur.key
-rw-r--r-- 1 root root 1519 Oct 30 11:44 serveur_chaine.crt
127.0.0.1 www.tp.local
```

4.2 Étape 2 : Configuration nginx et justification des directives

Directive	Justification
ssl_certificate (chaîne) et ssl_certificate_key	Présente la feuille et l'intermédiaire pour que le client reconstruise la chaîne jusqu'à une racine qu'il connaît
ssl_protocols TLSv1.2 TLSv1.3	Exclut SSLv3, TLS 1.0 et 1.1, obsolètes et vulnérables

Directive	Justification
ssl_ciphers 'ECDHE+AESGCM:ECDHE+CHACHA20'	En TLS 1.2, limite les suites à ECDHE (confidentialité persistante) avec chiffrement authentifié (AES-GCM, ChaCha20-Poly1305) ; TLS 1.3 n'a que des suites AEAD
ssl_prefer_server_ciphers off	Les suites modernes sont toutes sûres ; le client choisit celle qui lui convient (matériel, performances)
ssl_session_tickets off	Les clés de tickets ne sont pas renouvelées automatiquement : leur fuite déchiffrerait les sessions passées
ssl_client_certificate (racine)	Ancre de confiance pour valider les certificats clients
ssl_verify_client on	Certificat client obligatoire (authentification mutuelle)
ssl_verify_depth 2	Client, intermédiaire, racine : deux niveaux au-dessus de la feuille
ssl_crl crls.pem	Refuse les certificats révoqués ; doit contenir les CRL de l'intermédiaire **et** de la racine
add_header Strict-Transport-Security	Impose HTTPS au navigateur pendant un an (31 536 000 s)
return 200 "Bonjour $ssl_client_s_dn"	Affiche l'identité authentifiée du client

Vérification

La configuration a été testée sur nginx avec sudo nginx -t (« syntax is ok », « test is successful »), puis rechargée avec sudo systemctl reload nginx. Le comportement mutuel a été vérifié avec curl (réponse Bonjour CN=bob@tp.local) et openssl s_client.



4.3 Étape 3 : Observation de la poignée de main

Scénario	Résultat observé
Bob, TLS 1.3	TLSv1.3, suite TLS_AES_256_GCM_SHA384, groupe X25519MLKEM768, Verification: OK, réponse HTTP/1.1 200 OK : Bonjour CN=bob@tp.local
Bob, TLS 1.2 forcé	TLSv1.2, suite ECDHE-ECDSA-AES256-GCM-SHA384, Verification: OK, réponse 200 OK : Bonjour CN=bob@tp.local
Sans certificat client	HTTP/1.1 400 Bad Request : « No required SSL certificate was sent »
Alice (certificat révoqué)	HTTP/1.1 400 Bad Request : « The SSL certificate error » ; journal nginx : client SSL certificate verify error: (23:certificate revoked)

Avec curl -v, on voit la poignée de main TLS 1.3 complète (Client hello, Server hello, Encrypted Extensions, Request CERT, Certificate, CERT verify, Finished), puis SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768. Le certificat présenté est celui de www.tp.local, émis par Intermediaire TP, et le nom correspond au SAN (subjectAltName: "www.tp.local" matches).

Pour un client refusé, nginx ne coupe pas la connexion par une alerte TLS : la poignée de main se termine, puis il répond par une page HTTP 400. En TLS 1.3, le client voit en effet la connexion « établie » avant le rejet, car la vérification du certificat client se fait côté serveur à la fin de la poignée de main. Le motif du refus n'apparaît que dans le journal du serveur, au niveau info (error_log ... info).

Points d'attention.

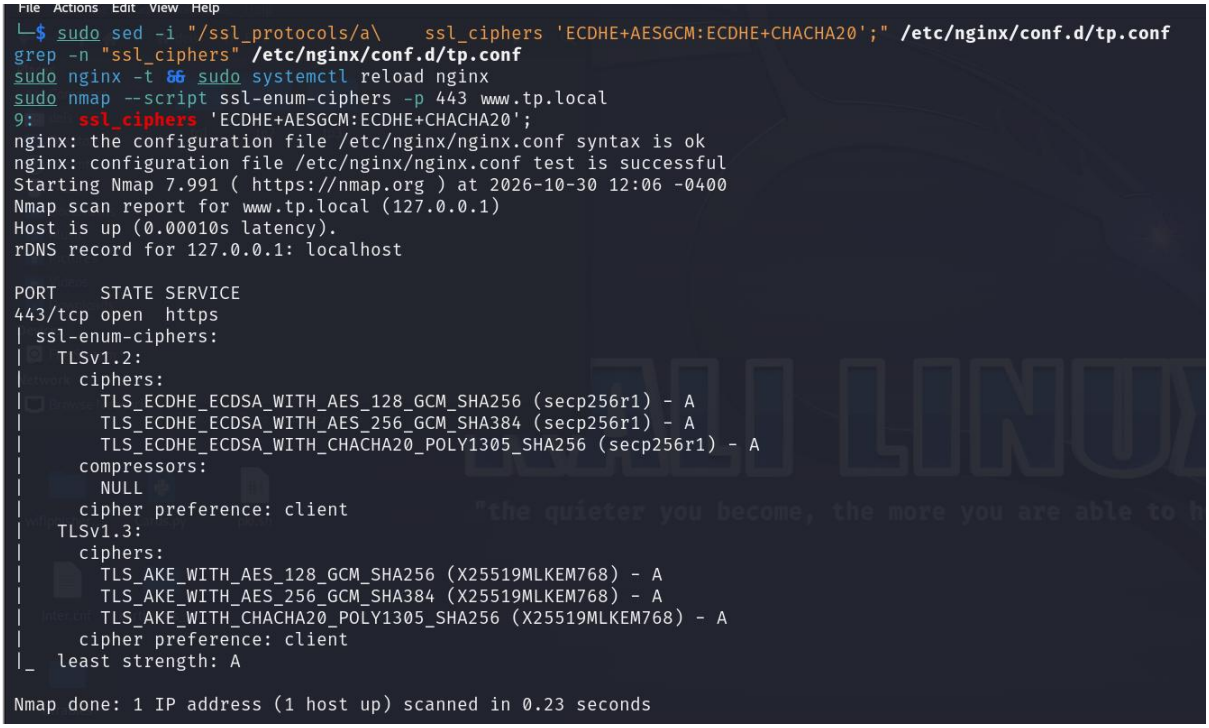
- **Chaîne du client avec s_client :** l'option -cert n'envoie que le premier certificat du fichier. Avec -cert bob_chaine.crt seul, nginx répond 400, car il ne connaît que la racine et ne peut pas reconstituer la chaîne. Il faut envoyer l'intermédiaire avec -cert bob.crt -cert_chain inter/inter.crt. curl, lui, envoie toute la chaîne du fichier bob_chaine.crt.
- **CRL :** avec la vérification des révocations, OpenSSL exige une CRL pour **chaque** niveau de la chaîne, racine comprise. C'est pourquoi crls.pem contient les deux listes : celle de l'intermédiaire et celle de la racine. Avec une seule, même un certificat valide serait refusé (unable to get certificate CRL).

Pourquoi le certificat du serveur n'est-il pas visible en TLS 1.3 dans Wireshark ? Le ClientHello et le ServerHello sont en clair (versions, suites proposées, parts d'échange de clés X25519MLKEM768, nom www.tp.local dans le champ SNI). Dès le ServerHello, client et serveur calculent un secret commun grâce à l'échange de clés : tout le reste de la poignée de main (EncryptedExtensions, Certificate, CertificateVerify, Finished) est déjà chiffré sous des clés de poignée. En TLS 1.2, au contraire, le certificat circule en clair. Sur la capture capture_tls.pcapng (32 paquets), la connexion TLS 1.3 montre un ClientHello, puis un ServerHello suivi uniquement d'enregistrements Application Data, et aucun message Certificate n'est lisible. La connexion TLS 1.2 montre au contraire le certificat du serveur (Server Hello, Certificate, Server Key Exchange, Certificate Request) et celui du client (Certificate, Client Key Exchange, Certificate Verify) en clair. Le chiffrement de la poignée de main en TLS 1.3 protège donc l'identité du serveur et celle du client, en authentification mutuelle, contre un observateur passif. Le ClientHello TLS 1.3 est volumineux (1563 octets), à cause de la part de clé de l'échange hybride résistant aux ordinateurs quantiques.

4.4 Étape 4 : Vérification de la configuration

Un client TLS 1.1 est refusé : le client OpenSSL moderne refuse même de le proposer (no protocols available) et, abaissé à SECLEVEL=0, le serveur répond par l'alerte 70 protocol version.

Le scan nmap --script ssl-enum-ciphers -p 443 www.tp.local a d'abord montré **13 suites TLS 1.2, dont 6 en mode CBC**, donc sans chiffrement authentifié. On a ajouté ssl_ciphers 'ECDHE+AESGCM:ECDHE+CHACHA20';, puis rechargé nginx. Après correction, TLS 1.2 ne propose plus que **3 suites AEAD** à confidentialité persistante (ECDHE avec AES-GCM ou ChaCha20-Poly1305), et TLS 1.3 propose ses 3 suites, toutes AEAD.



4.5 Étape 5 : Supervision des expirations

surveiller.py DOSSIER [--alerte 30] [--critique 7] lit tous les fichiers .crt, .pem et .cer du dossier (chaînes comprises), affiche pour chaque certificat le nombre de jours restants avant expiration, et rend un code de sortie utilisable par un outil de supervision : 0 si tout est OK, 1 si au moins un certificat est en alerte (moins de 30 jours), 2 si au moins un est critique (moins de 7 jours). Le test a été fait sur trois dossiers :

Dossier	Contenu	Résultat affiché	Code de sortie
test_ok	serveur.crt (90 jours)	OK, 89 j	0
test_alerte	moyen.crt (émis pour 20 jours)	ALERTE, 19 j	1
test_surv	court.crt (émis pour 5 jours) et serveur.crt	CRITIQUE, 4 j (et OK, 89 j pour serveur.crt)	2

Le code de sortie correspond au pire niveau rencontré dans le dossier. Le nombre de jours est arrondi vers le bas : un certificat émis pour 5 jours affiche 4 dès qu'il a perdu quelques secondes de vie, ce qui est le comportement prudent d'une supervision.

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp2]
└─$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out court.key
openssl req -new -key court.key -subj "/CN=court.tp.local" -addext "subjectAltName=DNS:court.tp.local" -out court.csr
openssl ca -batch -notext -config inter.cnf -extensions ext_serveur -days 5 -in court.csr -out court.crt
mkdir -p test_surv && cp court.crt serveur.crt test_surv/
python surveiller.py test_surv; echo "code de sortie : $?"
Using configuration from inter.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'court.tp.local'
Certificate is to be certified until Nov  4 16:19:24 2026 GMT (5 days)

Write out database with 1 new entries
Database updated
CRITIQUE     4 j  court.crt  [CN=court.tp.local]  expire le 2026-11-04
OK          89 j  serveur.crt  [CN=www.tp.local]  expire le 2027-01-28
code de sortie : 2
```

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp2]
└─$ mkdir -p test_ok && cp serveur.crt test_ok/
python3 surveiller.py test_ok; echo "code de sortie : $?"      # OK, code 0

openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out moyen.key
openssl req -new -key moyen.key -subj "/CN=moyen.tp.local" -addext "subjectAltName=DNS:moyen.tp.local" -out moyen.csr
openssl ca -batch -notext -config inter.cnf -extensions ext_serveur -days 20 -in moyen.csr -out moyen.crt
mkdir -p test_alerte && cp moyen.crt test_alerte/
python3 surveiller.py test_alerte; echo "code de sortie : $?"  # ALERTE, code 1
OK          89 j  serveur.crt  [CN=www.tp.local]  expire le 2027-01-28
code de sortie : 0
Using configuration from inter.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'moyen.tp.local'
Certificate is to be certified until Nov 19 16:21:01 2026 GMT (20 days)

Write out database with 1 new entries
Database updated
ALERTE      19 j  moyen.crt  [CN=moyen.tp.local]  expire le 2026-11-19
code de sortie : 1
```

```
┌──(venv)─(deis㉿deis)-[~/TP_EC1/tp2]
└─$ python3 surveiller.py /etc/nginx/tls; echo "code de sortie : $?"
OK        3649 j  racine.crt  [CN=Racine TP,O=Universite TP,C=BF]  expire le 2036-10-27
OK          89 j  serveur_chaine.crt  [CN=www.tp.local]  expire le 2027-01-28
OK        1822 j  serveur_chaine.crt#1  [CN=Intermediaire TP,O=Universite TP,C=BF]  expire le 2031-10-27
code de sortie : 0
```

Sur le dossier de déploiement /etc/nginx/tls, la supervision donne : racine 3649 j (expire le 2036-10-27), certificat du serveur 89 j (expire le 2027-01-28) et intermédiaire 1824 j (expire le 2031-10-29), tous OK, code de sortie 0.

4.6 Étape 6 : Renouvellement automatique

renouveler.sh enchaîne les cinq opérations demandées, précédées d'un test de seuil (openssl x509 -checkend) pour ne renouveler que si l'expiration est proche :

1. génération d'une nouvelle clé et d'une nouvelle demande ;
2. signature par l'intermédiaire ;
3. vérification avant tout déploiement (openssl verify avec nom et usage, et concordance entre la clé et le certificat) ;
4. archivage de l'ancien certificat dans archive/<horodatage>-<pid> (le PID évite qu'une archive en écrase une autre) ;

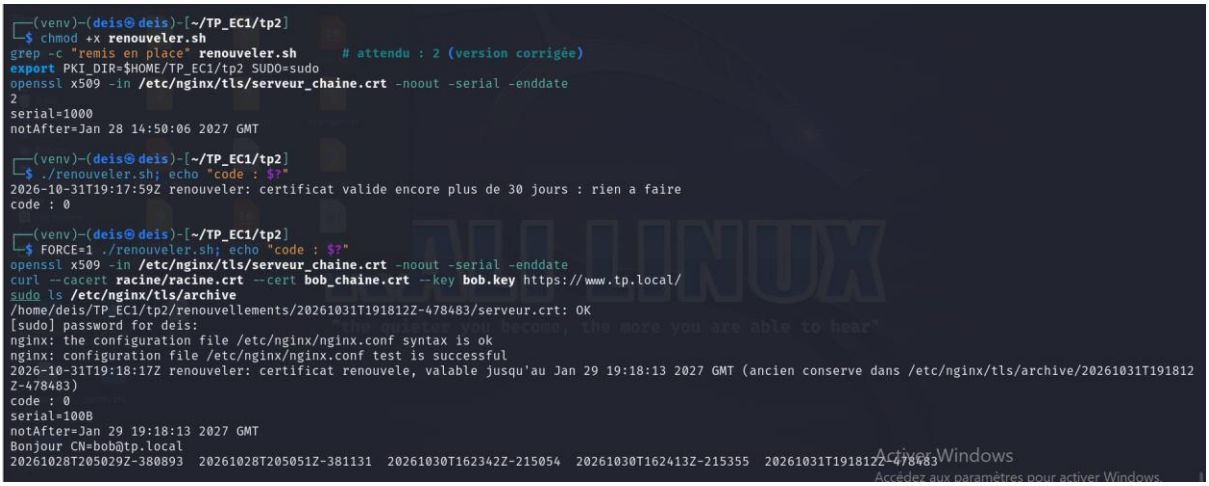
5. installation, nginx -t puis rechargement sans interruption, avec retour automatique à l'ancien certificat en cas d'échec.

Quatre scénarios ont été exécutés sur la machine de test (le renouvellement forcé avec le vrai nginx -t et le vrai rechargement, le test d'échec avec NGINX_TEST=false) :

Scénario	**Résultat**
Certificat encore valide plus de 30 jours	« rien à faire », code 0
Renouvellement forcé (FORCE=1, vrai nginx)	Série 1000 puis 100B ; verify OK ; nginx -t OK ; certificat valable jusqu'au 29 janvier 2027 ; rechargement sans coupure ; curl répond Bonjour CN=bob@tp.local ; ancien certificat archivé
Test de configuration en échec (NGINX_TEST=false)	« ancien certificat remis en place », code 1 ; série 100B avant et après ; curl répond toujours
Planification quotidienne	Ligne cron 17 3 * * * installée puis retirée (crontab -r)

Défaut trouvé et corrigé. Un premier essai avec la version initiale du script a révélé que le retour arrière n'avait aucun effet : après l'échec simulé, le certificat installé n'était pas l'ancien (série 1009 au lieu de 1008), alors que le script affichait « retour a l'ancien certificat ». Cause : le dossier d'archive est créé par root en mode 700 (umask 077), et le test [-d "$archive"] exécuté par un utilisateur ordinaire échouait, ce qui sautait la restauration. Correction : tous les tests passent par sudo, et le script vérifie le résultat avec cmp au lieu de le supposer. Nouveau test avec la version corrigée : même numéro de série avant et après l'échec (100B). Leçon : un test de retour arrière doit comparer l'état réel, pas se fier aux messages du script, et un script se teste avec les droits réels de son utilisateur.

La planification quotidienne est fournie dans renouveler.timer : un service et un minuteur systemd (OnCalendar=daily, Persistent=true), avec la variante cron en commentaire.





4.7 Questions de réflexion

1. **Pourquoi le serveur ne doit-il pas envoyer le certificat racine ?** Le client la possède déjà dans son magasin de confiance, et c'est ce qui fait sa valeur : une racine reçue du réseau ne prouverait rien, puisque n'importe qui pourrait en envoyer une. L'envoyer ajoute en plus des octets inutiles à chaque poignée. Le serveur n'envoie que la feuille et les intermédiaires.
2. **Quel est l'intérêt de désactiver les tickets de session lorsque leurs clés ne sont pas renouvelées ?** Le serveur chiffre l'état de la session dans le ticket avec une clé de ticket ; si cette clé n'est jamais renouvelée (c'est le cas par défaut dans nginx jusqu'à un redémarrage), quiconque la vole peut déchiffrer à posteriori tout le trafic enregistré qui l'a utilisée. Cela annule la confidentialité persistante offerte par ECDHE. Le prix est la perte de la reprise de session rapide, donc des poignées complètes plus coûteuses.
3. **Que garantit HSTS et contre quelle attaque protège-t-il ?** Il ordonne au navigateur de n'utiliser que HTTPS pour ce domaine pendant max-age (un an) : plus de requête HTTP, et impossibilité de passer outre une erreur de certificat. Il protège contre le SSL stripping (un attaquant en position d'homme du milieu qui réécrit la redirection HTTP vers

HTTPS) et contre le vol de cookies sur une connexion en clair. Limite : la toute première visite n'est pas protégée, d'où la liste de préchargement des navigateurs.

4. **Durée maximale des certificats publics : 100 jours en 2027, 47 jours en 2029.** Un renouvellement manuel devient impraticable : un seul certificat se renouvelle de 4 à 8 fois par an, et un parc de centaines de certificats devient ingérable à la main. Pour une organisation, cela impose d'automatiser (protocole ACME ou équivalent, scripts comme renouveler.sh), de tenir un inventaire de tous les certificats, de superviser les expirations (surveiller.py), de renouveler bien avant l'échéance, de prévoir un plan pour les équipements incapables de se renouveler seuls, et de pouvoir changer d'AC rapidement. Effet positif : une clé volée a une durée de vie utile plus courte.
5. **Un serveur expose un certificat valide mais la chaîne est incomplète : pourquoi certains clients l'acceptent-ils et d'autres non ?** Certains clients complètent eux-mêmes : ils ont déjà l'intermédiaire en cache (visite d'un autre site qui la présente) ou la téléchargent via l'extension AIA (Authority Information Access) du certificat. D'autres (curl, OpenSSL, Java, applications mobiles, clients d'API) ne le font pas et rejettent le certificat : le site « marche dans mon navigateur » mais pas dans un script. Il faut donc toujours servir la feuille et les intermédiaires, et tester avec un client strict comme openssl s_client.
6. **Procédure de réaction si la clé privée du serveur fuit.**
   - 6.1 Confirmer et dater la fuite ; couper si besoin l'accès à la clé (compte, sauvegardes, serveur).
   - 6.2 Générer immédiatement une **nouvelle** paire de clés : on ne réutilise jamais l'ancienne clé.
   - 6.3 Révoquer l'ancien certificat (openssl ca -config inter.cnf -revoke serveur.crt -crl_reason keyCompromise), puis regénérer et publier les CRL.
   - 6.4 Faire émettre et déployer le nouveau certificat (FORCE=1 ./renouveler.sh), vérifier la chaîne, puis redémarrer nginx pour renouveler aussi les clés de tickets.
   - 6.5 Rechercher la cause (journaux, accès, sauvegardes) et la corriger avant la remise en service : l'ancienne clé permet l'usurpation jusqu'à la révocation et l'expiration.
   - 6.6 Informer les personnes concernées et consigner l'incident. Grâce à ECDHE, le trafic passé n'est pas déchiffrable avec la seule clé du serveur.

5. Conclusion

Les trois TP forment une chaîne conséquente : le TP 1 montre comment assembler correctement dérivation de clés et chiffrement authentifié ; le TP 2 construit la confiance avec une PKI à deux niveaux, des contraintes qui limitent les dégâts d'une compromission et une révocation vérifiable ; le TP 3 met cette confiance au service d'un canal TLS mutuel, et montre qu'un certificat se gère dans la durée (supervision, renouvellement, réaction à l'incident).

Points à retenir :

- Une primitive correcte ne suffit pas : ce sont le nonce, les données associées et l'authentification de l'en-tête qui font la sécurité du format.
- Les contraintes (`pathlen`, `nameConstraints`, usages) sont contrôlées par celui qui valide, pas par celui qui signe.
- La révocation par CRL exige une liste par niveau de la chaîne ; avec des durées de vie de plus en plus courtes, l'automatisation devient obligatoire.

Tous les essais ont été réalisés sur la machine de test ; les sorties de terminal servent de preuves. Aucune clé privée ni mot de passe ne figure dans les livrables.

Table des matières

1. Introduction et environnement de travail 1
2. TP 1 : Chiffrement authentifié et dérivation de clés 1
2.1 Étape 1 : Dérivation des clés 1
2.2 Étape 2 : Format et fonctions chiffrer / déchiffrer 1
2.3 Étape 3 : Preuves par les tests 2
2.4 Étape 4 : Évolution des paramètres 3
2.5 Questions de réflexion 4
3. TP 2 : Construire une infrastructure de gestion de clés 5
3.1 Étape 1 : Rôle des lignes d'extensions 5
3.2 Étape 2 : AC racine 6
3.3 Étape 3 : AC intermédiaire 7
3.4 Étape 4 : Certificats de serveur et de client 7
3.5 Étape 5 : Validations observées 8
3.6 Étape 6 : Révocation 8
3.7 Étape 7 : Automatisation 9
3.8 Questions de réflexion 9
4. TP 3 : Déployer un canal sécurisé et gérer les certificats 11
4.1 Étape 1 : Préparation des fichiers 11
4.2 Étape 2 : Configuration nginx et justification des directives 11
4.3 Étape 3 : Observation de la poignée de main 13
4.4 Étape 4 : Vérification de la configuration 14
4.5 Étape 5 : Supervision des expirations 15
4.6 Étape 6 : Renouvellement automatique 16
4.7 Questions de réflexion 18
5. Conclusion 20